

3 Инструментальные средства разработки

- Сборка
 - Vite
- Ядро приложения
 - Vue 3 (composition API)
 - TypeScript
- Состояние и навигация
 - Pinia
 - Vue router
- Интерфейс
 - PrimeVue
 - Vue i18n
- Данные и формы
 - TanStack Query (Vue query)
 - Vee-validate и zod
- Качество и разработка
 - Vitest
 - Storybook
 - ESLint и Prettier
 - Husky
- Окружение и развёртывание
 - JSON-server
 - Docker и nginx
- Описание результатов

3 Инструментальные средства разработки

Перед рассмотрением прикладных задач, которые были решены в рамках работы, необходимо сначала описать выбранные инструментальные средства разработки. Это поможет читателю понять, какие технологии были использованы для решения задач и какие возможности были доступны для автора работы. Также это поможет другим специалистам в данной области оценить применимость выбранных инструментов и возможность их использования в своих проектах.

Для реализации интерфейса, управления данными, маршрутизации и сопровождения жизненного цикла разработки был сформирован программный стек на базе экосистемы

Vue 3. Материал главы организован по функциональным блокам: платформа сборки, ядро приложения, управление состоянием и навигация, построение интерфейса, работа с данными и формами, инструменты качества кода, а также окружение разработки и развертывания.

Описание каждой технологии строится по единому шаблону:

*1. **Назначение** — краткая характеристика инструмента и его роли в современной frontend-разработке.*

*2. **Решаемые проблемы** — какие задачи проекта обусловили необходимость его применения.*

*3. **Обоснование выбора** — сравнение с рассмотренными альтернативами и аргументы в пользу принятого решения.*

*4. **Применение в проекте** — конкретные места использования в кодовой базе (модули, сценарии, команды).*

*5. **Иллюстрации** (при необходимости) — схемы, таблицы или скриншоты, поясняющие интеграцию инструмента.*

Сводная характеристика всего стека приведена в таблице 3.1 в конце главы.

3.1. Сборка

3.1.1 Vite

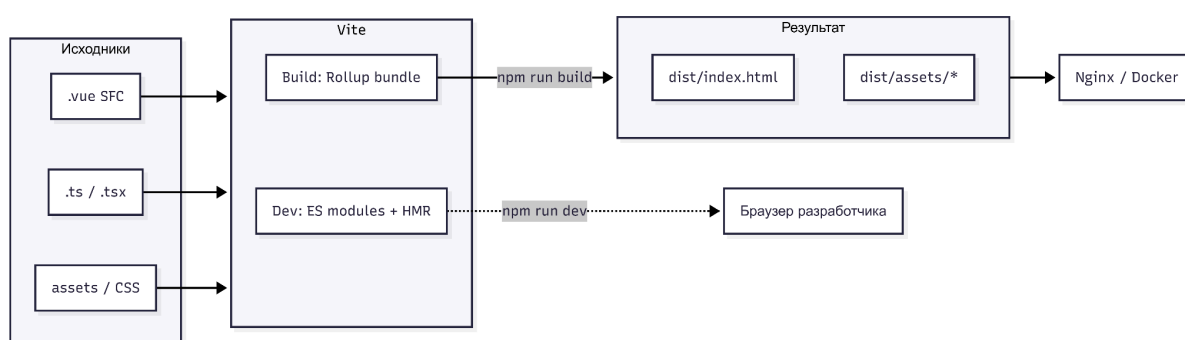
Vite — инструмент сборки и dev-сервера для современных фронтенд-приложений. В режиме разработки он использует нативные ES-модули браузера и подает исходный код по необходимости, не собирая всё приложение целиком при каждом старте. Для production выполняется оптимизированная сборка на базе инструмента Rollup.

Решаемые проблемы. Без специализированного сборщика пришлось бы вручную настраивать транспиляцию TypeScript, обработку .vue-файлов, hot module replacement (HMR), алиасы путей и проксирование API. Классические связки Webpack + loader'ы дают более медленный цикл разработки на проектах.

Обоснование выбора. Vite официально рекомендуется для проектов Vue 3, хорошо интегрируется с TypeScript, Vitest и Storybook. Альтернатива:

- Webpack (Create Vue App) — уступает по скорости dev-сервера
- Nuxt избыточен для SPA без SSR.

Применение в проекте. Конфигурация задаётся в файле vite.config.ts. Плагин @vitejs/plugin-vue для SFC, алиас @/ → src/, proxy /api на бэкэнд (localhost:8080) с удалением префикса — поведение, повторённое позже в nginx для production. Сборка выполняется командой npm run build, результат помещается в каталог dist/.



3.2. Ядро приложения

3.2.1. Vue 3 (composition API)

Vue 3 — прогрессивный фреймворк для построения пользовательских интерфейсов. *Composition API* — способ организации логики компонента через функции (*setup*, *composables*) вместо опций объекта (*Options API*).

Решаемые проблемы. *Composition API* упрощает выделение переиспользуемой логики (авторизация, работа с формами, запросы), улучшает типизацию в TypeScript и делает код крупных компонентов читаемым: связанная логика группируется рядом, а не разносится по блокам *data*, *methods*, *computed*.

Обоснование выбора. Vue 3 выбран как основной UI-фреймворк проекта; *Composition API* используется во всех компонентах (`<script setup lang="ts">`). React потребовал бы иной экосистемы и архитектурных решений; Angular — более высокого порога входа для приложения учебной платформы.

Применение в проекте. Интерфейсы трёх ролей (администратор, преподаватель, студент) реализованы как набор Vue-компонентов с декларативными шаблонами. Логика вынесена в *composables* и *Pinia-stores*. Макросы Vue 3.4+ (*defineProps*, *defineEmits*, *defineModel*) применяются для типобезопасного API компонентов.

3.2.2. TypeScript

TypeScript — типизированное надмножество JavaScript (рис. 1), компилируемое в обычный JS. Ошибки типов выявляются на этапе разработки и сборки, а не только в *runtime*.

Решаемые проблемы. В проекте с десятками сущностей (пользователь, курс, *assessment*, организационная иерархия) без статической типизации высок риск несогласованности данных между API, *store* и UI. TypeScript фиксирует контракты: DTO ответов бэкенда, доменные типы, *props* компонентов.

Обоснование выбора. TypeScript — стандарт фронтенд приложений; поддерживается *vue-tsc*, *Vite*, *ESLint*. В конфигурации включён режим *noUncheckedIndexedAccess*, что требует явной проверки индексов массивов и ключей объектов — повышает надёжность при работе с динамическими данными.

Применение в проекте. Типы сущностей описаны в слое *entities* (*User*, *Course*, *StudentCourse* и др.). API-ответы мапятся в доменные модели (*mapAdminUserListItemToUser*). Абсолютные импорты через *@/* согласуются с *path mapping* в *tsconfig.app.json*.

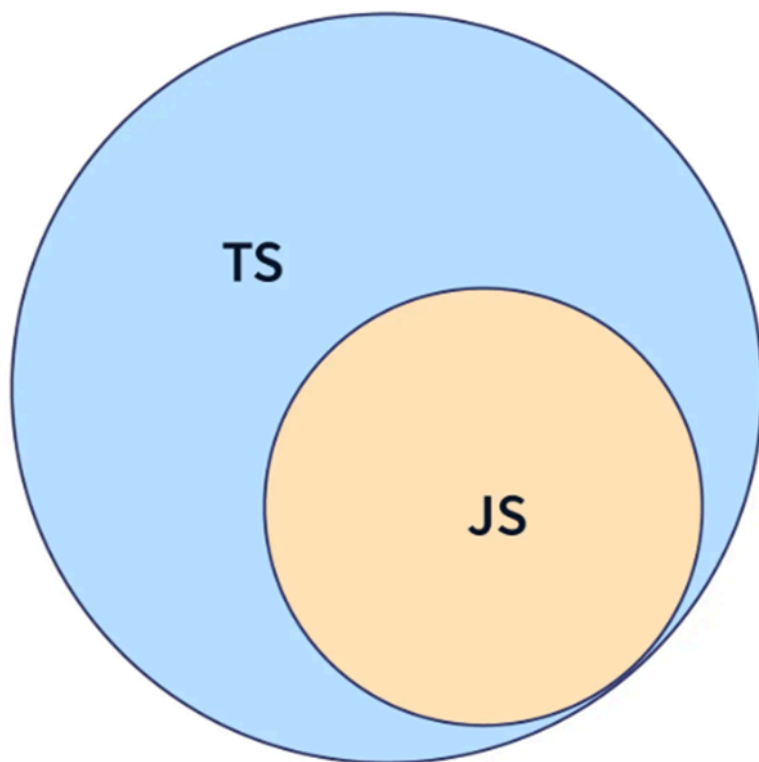


Рис. 2. Соотношение языков JavaScript и TypeScript

3.3. Состояние и навигация

3.3.1 Pinia

Pinia — официальное решение для управления состоянием в Vue 3, преемник Vuex с более простым API и полной поддержкой TypeScript.

Решаемые проблемы. Часть состояния должна быть доступна глобально: сессия пользователя, тема оформления, локальные UI-флаги. Передача таких данных через props по всему дереву компонентов не масштабируется.

Обоснование выбора. Pinia рекомендован документацией Vue 3; stores описываются как composable-функции (defineStore + setup-синтаксис), что согласуется с Composition API. Vuex устареваает; Context/Provide подходит только для узких сценариев.

Применение в проекте. useSessionStore хранит факт авторизации и роль пользователя (из JWT). useThemeStore управляет светлой/тёмной темой. Дополнительные stores в слое entities инкапсулируют состояние отдельных доменных областей. Серверные данные (списки курсов, пользователей) сознательно не дублируются в Pinia — для них используется TanStack Query (п. 3.5.1).

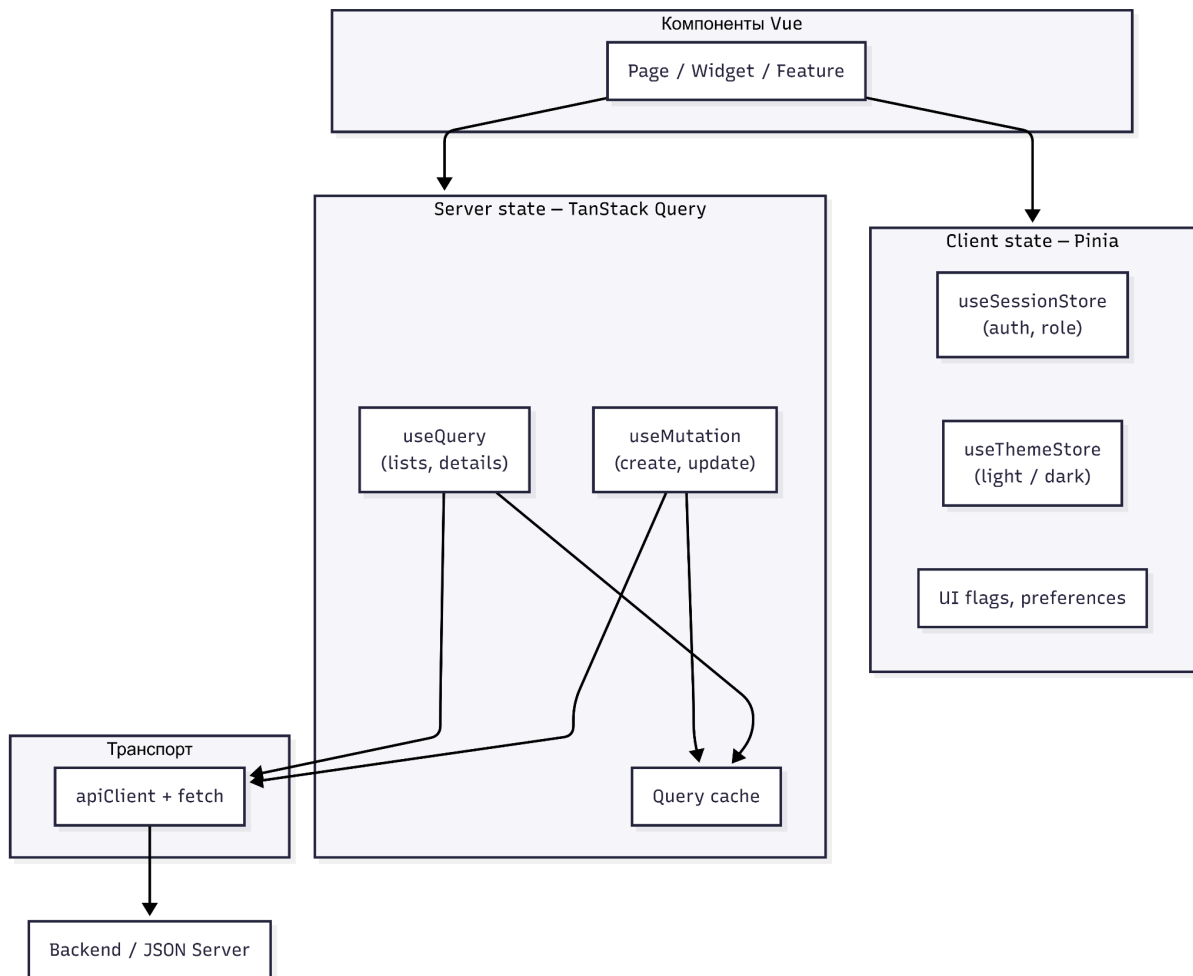


Рисунок 3.4 — Схема разделения: «Pinia — клиентское состояние / Vue Query — серверное состояние».

3.3.2 Vue router

Vue Router — официальная библиотека маршрутизации для Vue, обеспечивающая навигацию в SPA без перезагрузки страницы.

Решаемые проблемы. Платформа имеет три изолированных «портала» (/admin, /teacher, /student) с вложенными маршрутами (курсы, профили, прохождение тестов). Нужны lazy loading страниц, защита маршрутов по авторизации и роли (RBAC), редирект после логина.

Обоснование выбора. Vue Router интегрирован с Vue 3, поддерживает nested routes, navigation guards, programmatic navigation. Альтернативы (сторонние router'ы) не дают преимуществ в экосистеме Vue.

Применение в проекте. Маршруты объявлены в `app/providers/router/index.ts`. Глобальный `beforeEach guard` (`guards.ts`) проверяем `requiresAuth`, `requiresGuest` и `allowedRoles` на основе `useSessionStore`. Студентский flow включает цепочку: курс → информация об `assessment` → сессия прохождения (`student-assessment-take`).

3.4 Интерфейс

3.4.1 PrimeVue

PrimeVue — библиотека UI-компонентов для Vue с набором готовых элементов: таблицы, формы, модальные окна, уведомления, деревья и др.

Решаемые проблемы. Разработка админ-панели, таблиц пользователей, форм создания курса и assessment UI «с нуля» потребовала бы значительных трудозатрат и не гарантировала единообразия и базовой доступности (a11y).

Обоснование выбора. PrimeVue 4 поддерживает Vue 3, темизацию приложения через design tokens, тёмную тему через darkModeSelector. Альтернативы: Vuetify (Material Design, другая визуальная система), Element Plus (менее гибкий theming). PrimeVue выбран за богатый набор «enterprise»-компонентов, подходящих для учебной платформы.

Применение в проекте. Глобально подключены PrimeVue, ToastService, ConfirmationService в main.ts. Компоненты Button, InputText, DataTable, Select, Tag, Dialog используются в features и widgets. Тема Aura согласована с собственными design tokens (tokens.css); переключение light/dark синхронизировано с useThemeStore.

3.4.2 Vue i18n

Vue I18n — библиотека интернационализации для Vue: перевод строк, переключение локали.

Решаемые проблемы. Интерфейс платформы должен поддерживать как минимум русский и английский языки. Хардкод строк в шаблонах затрудняет поддержку и расширение локалей.

Обоснование выбора. Vue I18n — стандарт de facto для Vue; интегрируется с Composition API (useI18n). Альтернатива — ручные словари без библиотеки — не масштабируется.

Применение в проекте. Конфигурация в shared/config/i18n: файлы en.json, ru.json. Компонент LanguageSwitcher переключает локаль и сохраняет выбор в localStorage. Дополнительно синхронизируется локаль PrimeVue (календари, фильтры) через getLocaleMessage.

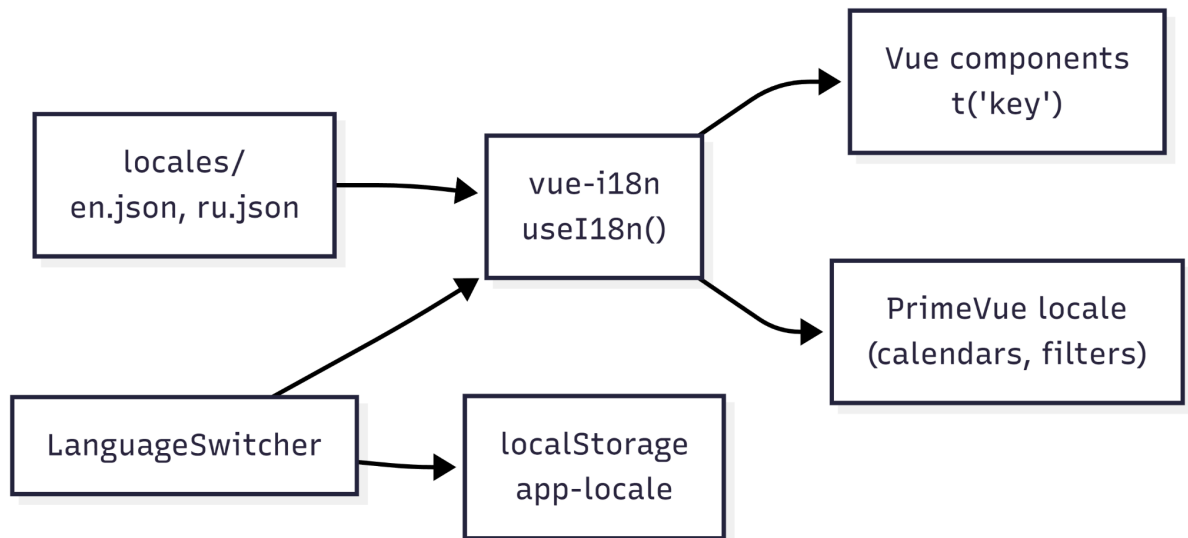


Рисунок 3.8 — Схема: «файлы локалей → vue-i18n → компонент / PrimeVue locale».

3.5 Данные и формы

3.5.1 TanStack Query (Vue query)

TanStack Query (Vue Query) — библиотека для управления серверным состоянием: запросы к API, кэширование, фоновое обновление, обработка loading/error.

Решаемые проблемы. Без централизованного слоя каждый компонент самостоятельно вызывал бы fetch, дублировал бы логику загрузки и не переиспользовал бы уже полученные данные. При навигации между страницами курсов это приводило бы к лишним запросам и «миганию» UI.

Обоснование выбора. TanStack Query — стандарт для работы с серверными данными в клиентском приложении; Pinia при этом остаётся для client state. SWR/React Query аналогичны, но TanStack имеет официальный Vue-адаптер. «Голый» fetch + Pinia для всех данных смешивал бы два типа состояния в одном store.

Применение в проекте. VueQueryPlugin подключён в main.ts. API-функции в слое entities/*api вызываются из queryFn / mutationFn. Devtools включены для отладки. Списки курсов, пользователей, банков вопросов — типичные сценарии использования.

3.5.2 Vee-validate и zod

Vee-Validate — библиотека валидации форм для Vue. **Zod** — декларативная схема валидации и парсинга данных на TypeScript с выводом типов.

Решаемые проблемы. Формы платформы (авторизация, добавление пользователя, создание курса) требуют проверки полей на клиенте до отправки на сервер. Разрозненные if в обработчиках submit сложно поддерживать; правила валидации должны совпадать с типами payload для API.

Обоснование выбора. Связка @vee-validate/zod позволяет описать схему один раз в Zod и использовать её и для валидации UI, и для типизации (z.infer). Альтернатива — только Vee-Validate с rule-объектами без единой схемы — хуже для сложных форм с условными правилами (например, обязательность departmentId только для роли Teacher).

Применение в проекте. Пример: feature add-user — createAddUserFormSchema(t) в schema.ts с superRefine для role-specific правил; buildCreateAdminUserPayload формирует

тело запроса после успешной валидации. Unit-тесты покрывают и схему, и сборку payload.

3.6 Качество и разработка

3.6.1 Vitest

Vitest — *test runner*, ориентированный на проекты Vite: общая конфигурация, быстрый запуск, API, совместимый с Jest.

Решаемые проблемы. Необходимо автоматически проверять корректность бизнес-логики (мапперы, схемы, сервисы) при изменениях кода. Ручное тестирование регрессий в проекте с десятками экранов трудоемко.

Обоснование выбора. Vitest использует тот же *pipeline*, что Vite; не требует отдельной тяжелой конфигурации Jest для ESM и Vue. Окружение *happy-dom* даёт DOM API для тестов, зависящих от окружения браузера, например, *LocalStorage*.

Применение в проекте. Конфигурация в *vitest.config.ts* с проектом *unit* (*unit-тесты*) и проектом *storybook* (тесты *stories* через *Playwright*).

3.6.2 Storybook

Storybook — среда для изолированной разработки и документирования UI-компонентов вне основного приложения.

Решаемые проблемы. Компонент внутри страницы сложно разрабатывать и демонстрировать: нужна авторизация, навигация, загрузка данных. Storybook позволяет рендерить компонент с *mock-props* и глобальными провайдерами (*Pinia*, *i18n*, *PrimeVue*).

Обоснование выбора. Storybook — стандарт для *design systems* и *component-driven development*. Интеграция с Vue 3 + Vite официально поддерживается. Аддоны *@storybook/addon-a11y* и *@storybook/addon-vitest* закладывают основу для *accessibility*- и *visual*-тестирования без отдельного E2E на раннем этапе.

Применение в проекте. *Stories* расположены рядом с компонентами (*WidgetCard.stories.ts*, *CourseCard.stories.ts* и др.). В *.storybook/preview.ts* подключены *design tokens* (*tokens.css*), *PrimeVue*, *i18n* — визуально *stories* соответствуют приложению.

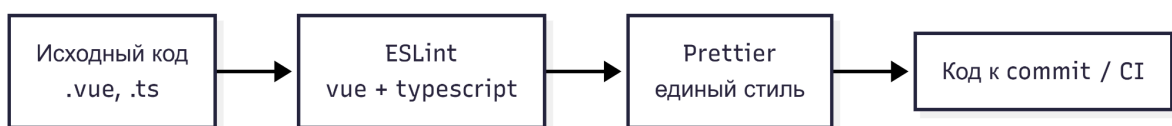
3.6.3 ESLint и Prettier

ESLint — линтер для JavaScript/TypeScript; **eslint-plugin-vue** — правила для Vue SFC.
Prettier — форматтер кода/файлов.

Решаемые проблемы. В командной разработке важны единообразие стиля и раннее обнаружение ошибок (неиспользуемые переменные, неверное использование Vue API). Без автоматизации качество кода зависит только от дисциплины разработчика.

Обоснование выбора. Связка ESLint + Prettier — стандарт Vue/TypeScript проектов. `eslint-config-prettier` исключает конфликты правил форматирования.

Применение в проекте. Скрипт `npm run lint` запускает `eslint` с `--fix`. `npm run format` форматирует `src/` через Prettier. Конфигурация ESLint — `eslint.config.ts` (flat config).



3.6.4 Husky

Husky — инструмент для настройки Git hooks: скрипты, автоматически выполняемые при `commit`, `push` и др.

Решаемые проблемы. Без hooks разработчик может закоммитить код с ошибками линтера, непрогнанными тестами или неверным форматированием.

Обоснование выбора. Husky — наиболее распространённое решение для проектов; проще альтернативных решения в виде ручных pre-commit скриптов или Lefthook. Планируется связка с `lint-staged` для проверки только изменённых файлов — сокращение времени hook'a.

Применение в проекте. Hook pre-commit: `lint-staged` → Prettier + ESLint на staged-файлах; опционально `npm run test:run` на pre-push. Это замыкает цепочку качества: локально (Husky) → CI → production.

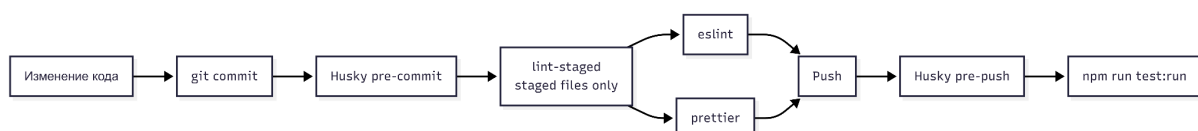


Рисунок 3.13 — Схема: «git commit → Husky pre-commit → lint-staged → lint/format/test».

3.7 Окружение и развёртывание

3.7.1 Json-server

JSON Server — утилита для быстрого развёртывания REST API поверх файла JSON. Она интерпретирует коллекции в `db.json` как ресурсы (`/users`, `/courses` и т.д.) и автоматически поддерживает операции `GET`, `POST`, `PUT`, `PATCH`, `DELETE`, а также фильтрацию и пагинацию через *query*-параметры.

Решаемые проблемы. На ранних этапах создания клиентской части платформы полноценный бэкенд может быть недоступен, нестабилен или реализован частично. При этом *frontend* уже должен:

- отображать списки пользователей, курсов, организационную иерархию;
- обрабатывать формы создания и редактирования (`POST/PATCH`);
- обрабатывать сценарии авторизации и обработки ошибок сети.

Без *mock API* разработчик вынужден либо определять данные в компонентах, либо писать объёмный перехватчик запросов внутри клиентского кода. Оба подхода усложняют поддержку и не соответствуют реальному HTTP-взаимодействию. *JSON Server* выносит *mock*-данные в отдельный процесс, что позволяет клиентской части работать с тем же *fetch* или *TanStack Query*, что и в *production*, меняя только адрес API через механизм *проху* в сборщике *Vite*..

Обоснование выбора.

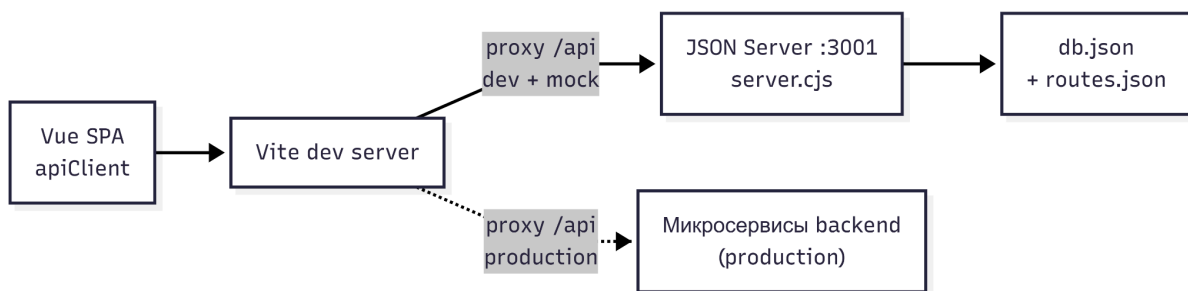
Критерий	<i>JSON Server</i>	<i>MSW (Mock Service Worker)</i>	<i>Mock</i> в коде клиента
Где выполняется <i>mock</i>	Отдельный <i>Node.js</i> -процесс	<i>Service Worker</i> в браузере	Внутри <i>apiClient</i>
REST из коробки	Да	Нужно описывать <i>handlers</i>	Нужно писать ручную
Редактирование данных	<code>db.json</code> + сохранение при <code>POST</code>	<i>In-memory</i> в <i>handlers</i>	<i>In-memory</i> в коде
Близость к реальному API	Высокая	Высокая	Средняя

Сложность настройки	Низкая	Средняя	Растёт с числом эндпоинтов
------------------------	--------	---------	-------------------------------

MSW удобен для тестов и изоляции компонентов, но для разработки экранов с десятками эндпоинтов JSON Server проще сопровождать: достаточно обновить JSON-файл.

Mock в коде клиента (перехват в `apiClient`) смешивает инфраструктуру и бизнес-слой, увеличивает размер бандла и усложняет переключение на реальный бэкэнд.

Вывод. JSON Server выбран как основной инструмент локального mock API на этапе разработки frontend.



3.7.2 Docker и nginx

Docker — платформа контейнеризации; **Nginx** — веб-сервер для раздачи статики и reverse proxy.

Решаемые проблемы. Production-сборка должна воспроизводимо собираться в одинаковом окружении и раздаваться с поддержкой SPA-маршрутизации (fallback на index.html) и проксирования API.

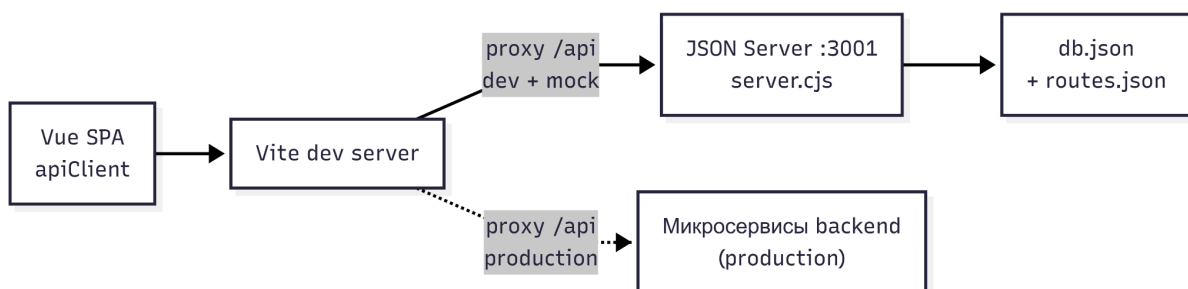


Рисунок 3.15 — Multi-stage Dockerfile (build → nginx).

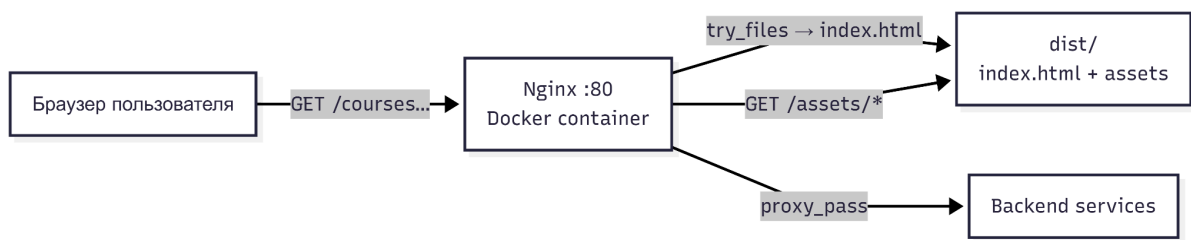


Рисунок 3.16 — Схема deployment: «браузер → nginx:80 → static | /api → backend:8080».

3.8 Итог

В данном пункте представлена сводная таблица используемого программного стека

№	Технология	Категория	Назначение в проекте	Рассмотренные альтернативы	Причина выбора
1	Vite	Сборка	Dev-сервер, HMR, production build	Webpack, Parcel	Скорость разработки, нативная поддержка Vue/TS
2	Vue 3	UI-фреймворк	Компоненты, Composition API	React, Angular	Опыт использования, согласованная экосистема
3	TypeScript	Типизация	DTO, domain types, props	JavaScript	Раннее обнаружение ошибок, упрощение рефакторинга
4	Pinia	Client state	Сессия, тема, UI-состояние	Vuex, composables-only	Официальный стандарт Vue 3, TS поддерживается из коробки
5	Vue Router	Маршрутизация	RBAC, три портала ролей	-	Стандарт для Vue
6	PrimeVue	UI-kit	Таблицы, формы, модальности, Toast	Vuetify, Element Plus	Богатый набор компонентов, доступна кастомизация при помощи css токенов
7	Vue I18n	Локализация	RU/EN	ручные	Стандарт

			интерфейс	словари	для Vue
8	TanStack Vue Query	Server state	Кэш API, mutations, loading/error	SWR, ручной fetch + Pinia	Разделение client/server state. Уменьшение написания однотипной логики
9	Vee-Validate + Zod	Валидация форм	Unit-тесты логики	Yup, только Vee-Validate	Единая Zod-схема + вывод типов
10	Vitest	Тестирован ие	Unit-тесты логики и компоненто в	Jest	Общий pipeline с Vite
11	Storybook	UI-документ ация	Stories, a11y, visual-тест ы	Ladle, Histoire	Экосистема , addon-vitest
12	ESLint + Prettier	Качество кода	линтинг и форматтер	Biome	Стандарт индустрии
13	Husky	Git hooks	Pre-commit/p re-push проверки	самописные проверки	Стандарт проектов
14	JSON Server	Dev-окружен ие	REST mock API из JSON	MSW, mock-client в коде	Отдельный сервер, CRUD из коробки
15	Docker + Nginx	Deploy	Сборка образа, SPA, proxy /api	-	Стандарт индустрии